1. ***When designing high-traffic microservices, what are your go-to strategies for ensuring resilience when a downstream dependency begins lagging or failing?***
    a. Goal
        i. My goal would be to prevent system wide outages while preserving the best possible user experience.
    b. Solution
        i. To detect service degradation it is essential to capture key metrics such as latency, timeouts, error rates, success rates and resource utilization CPU/Memory. These metrics should be collected at both the pod level and centrally.
        ii. To handle downstream lagging
            1. I will do retries with exponential backoff and jitter for transient failures.
            2. If the system continues to degrade, then open a circuit breaker to fast-fail subsequent requests until the dependency recovers.
        iii. To handle downstream unavailable
            1. I will handle depending on business requirements and the consistency model
            2. For eventual consistency, Serve cached data, degrade non-critical functionality or queue requests for asynchronous processing.
            3. For strong consistency fail fast with clear error responses.
        iv. I also use the bulkhead pattern to isolate resources such as connection pools for different downstream services to prevent resource exhaustion and impact on unrelated requests.
        v. And, if the degradation is caused by increased traffic or resource saturation than application failures for that I will have horizontal autoscaling in place to increase capacity and maintain service availability.

2. ***We use the Model Context Protocol (MCP) to connect internal IT data with LLMs. If you were designing a server to expose a sensitive database to an AI agent, how would you approach security and reduce the risk of schema misinterpretation?***
    a. Goal
        i. My goal would be to provide rich contextual framing while enforcing strict data access rules.
    b. Solution
        i. For Security, I will
            1. Authenticate AI agents using OAuth and enforce least privilege authorization for every MCP resource
            2. Provide read only access wherever possible
            3. Apply fine grained access controls like Row Level Security for db etc.
            4. Keep a Human in the Loop for any write, update, or delete operations
            5. Log and Audit every request.

6. Apply rate limiting to prevent abuse
7. Do Input validation and have parameterized queries.

ii. To Reduce schema misinterpretation, I will
1. Expose curated business resources instead of raw backend objects or schemas
2. Expose these resources as read only MCP Resources wherever possible
3. Define richly documented JSON Schemas with clear field descriptions,data types, constraints.
4. Use MCP Prompts to provide system instructions and few shots.
5. Do Input and output validation to ensure all requests and responses conform to the defined schema
6. Use semantic guardrails to detect malicious intent

3. As a technical lead, how would you help a team adopt Agentic IDEs or AI-assisted coding tools while maintaining long-term code quality and deep system understanding?

a. Goal
   i. My goal would be to ensure velocity gains and maintain code quality and ownership.

b. Solution
   i. Add repository level instruction files, like cursor rules and prompt templates for common tasks.
   ii. Strengthen CI/CD pipelines to automatically flag AI-generated anti-patterns or vulnerabilities by adding automated linting, mandatory test coverage thresholds etc.
   iii. Mandate Design, API Specs are well written and reviewed before AI assisted coding.
   iv. Code must have Minimum Test coverage numbers to merge PR.
   v. For Code review
      1. Build PR review peer team for each PR
      2. Code walkthrough by submitter to review if needed and explain code well to make sure code purpose is known and understood.
   vi. Sync up and Knowledge sharing in dedicated meeting or shared doc
      1. Share effective prompting workflows
      2. Or, where AI assistance failed or introduced subtle bugs
      3. Iterate on team best practices.

4. ***How do you diagnose a performance bottleneck in a containerized and orchestrated environment? What steps do you take to determine whether the issue stems from application logic, container resource limits, or networking?***
    a. I will take following steps
        i. Check system level dashboards in tools like Prometheus, Grafana or similar platforms to view cluster health. I will look for spikes in CPU, Memory, Network, Pod restart, number of instances.
        ii. Check application logs in Splunk or a similar, in affected timeframe for exceptions, errors etc.
        iii. Check the lifecycle of impacted requests using distributed tracing ID.
        iv. After identifying the affected component, will investigate the component, checking code for inefficient business logic.
        v. If code is okay, then check slow downstream services like slow db, cache miss, request drop due to rate limit etc.
        vi. Also, check service to service latency, by comparing client request dispatch and server receipt to identify underlying infra related delay, could be due to DNS, Load Balancer etc
        vii. Based on Co-related logs, metrics throughout and across systems finalise RCA.

5. ***Describe a time you made a significant architectural trade-off to meet a deadline. How did you manage the technical debt afterward to prevent long-term impact?***
    a. Context
        i. While working on Adobe Express which is an orchestration workflow integrating backend services across Document Cloud and Creative Cloud performance profiling revealed several optimizations like upload batching, parallel service initialization, asynchronous analytics, and ZIP based processing.
        ii. However implementing all of them required cross-team design approvals and would have delayed our release deadline
    b. Action
        i. To meet the deadline without compromising the long-term architecture I split the work into two phases
        ii. Phase 1
            1. I implemented low-risk, backward-compatible optimizations such as asynchronous analytics and ZIP-based processing.
            2. These changes delivered immediate performance improvements and could be shipped within the release timeline without requiring cross-team design changes
        iii. Phase 2
            1. In parallel, I started driving the larger architectural improvements, including upload batching and parallel service initialization.
            2. Since these changes impacted shared components across Acrobat and Creative Cloud, they required cross-team design

discussions, approvals, and coordinated implementation, so they were completed after the initial release

iv. Result

1. The team delivered the release on schedule, and the Phase 2 improvements reduced p50 latency from approximately 16 seconds to 8 seconds, providing a more scalable architecture for future growth.